

Introduction to Lab 2

SE4AI for Society, Learning to Find Trade-Offs



sesa^{lab}
SOFTWARE ENGINEERING
SALERNO



Gianmario Voria

Ph.D. Student
Software Engineering (SeSa) Lab
University of Salerno

The European Union Comes to Rescue!

The Artificial Intelligence Act represents the first law on AI by a major regulator anywhere - like the EU's GDPR in 2018, the EU AI Act could become a global standard.

AI-systems will be classified according to their risk levels. A system will be classified as *unacceptable*, and therefore *banned*, in the following cases:

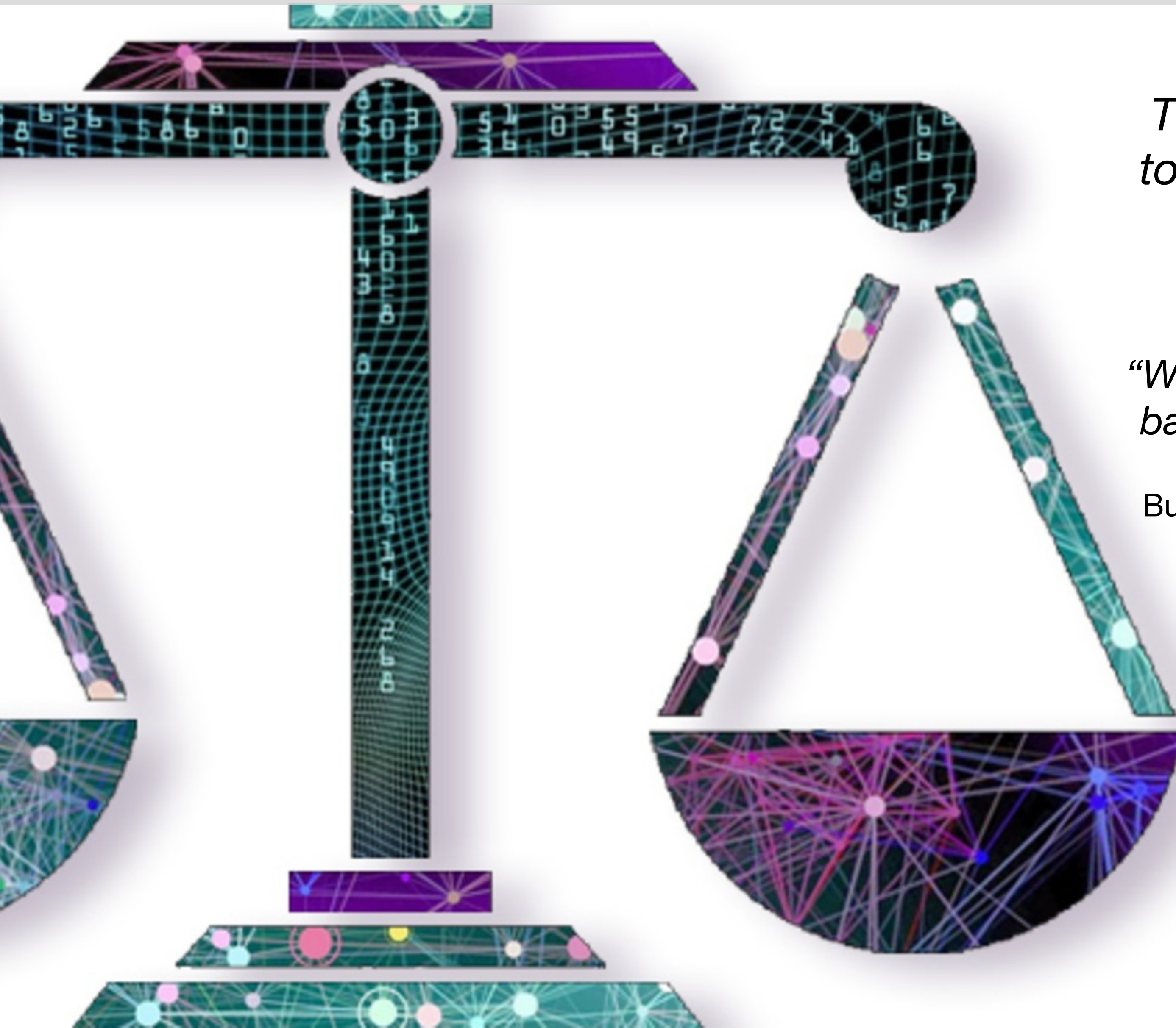
- (1) *Cognitive behavioral manipulation of people or specific vulnerable groups;*
- (2) *Social scoring: classifying people based on behavior, socio-economic status or personal characteristics;*
- (3) *Real-time and remote biometric identification systems.*

These properties have all to do with a specific non-functional requirement: *software fairness*!



Introduction to Lab 2 - SE4AI for Society, Learning to Find Trade-Offs

Machine Learning Fairness



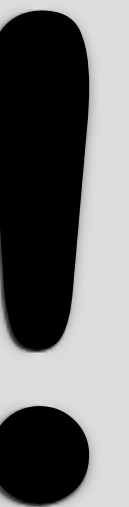
The research branch aiming at defining methods to correct algorithmic bias in automated decision processes based on machine learning.

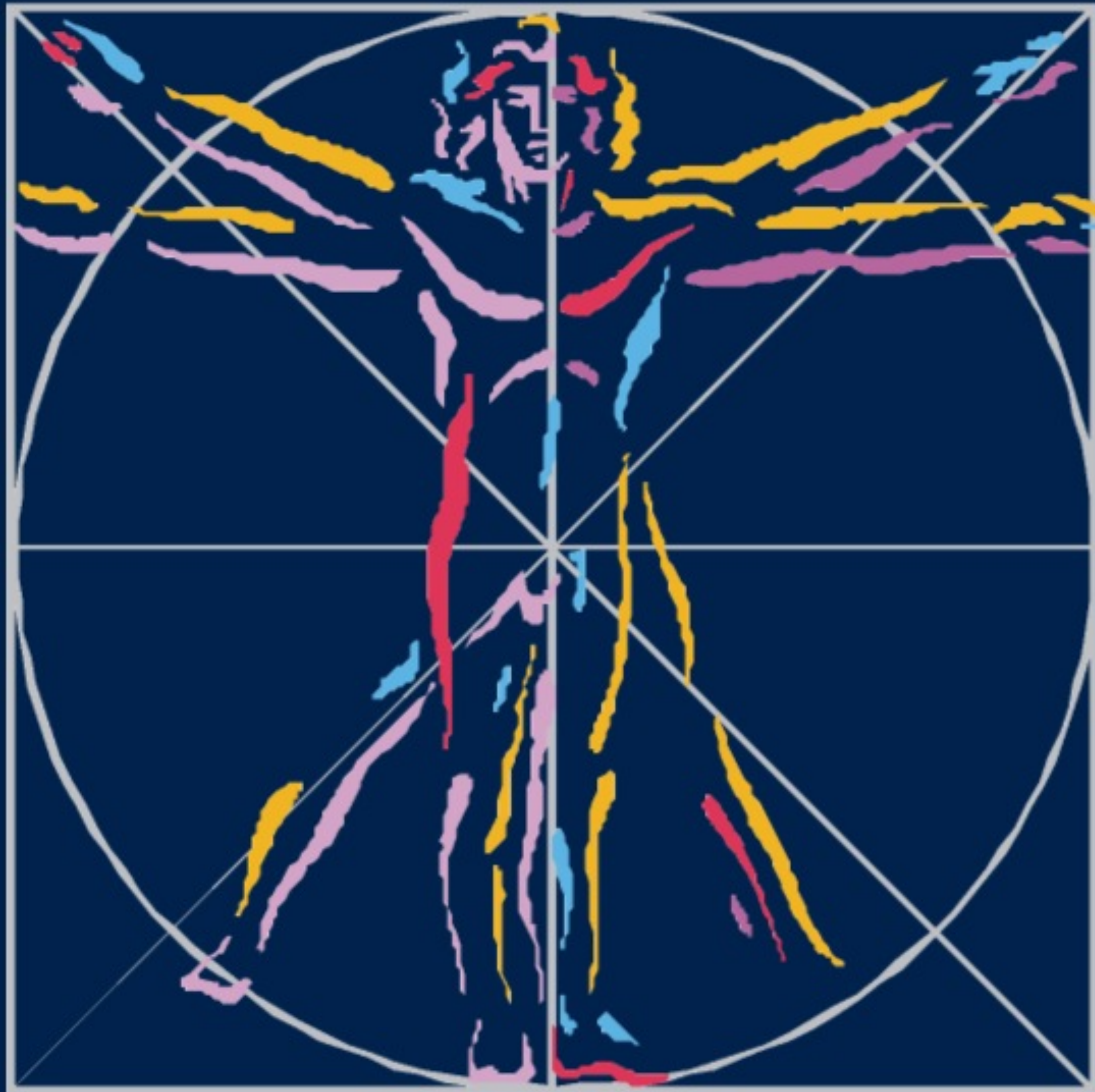
“When you think of A.I., it’s forward-looking. But A.I. is based on data, and data is a reflection of our history.”

Buolamwini, “Coded Bias”, 2020

Long story short

Voluntarily or not, data can perpetuate society’s existing race-, class- and gender-based inequities





It is, by nature, data-driven

Depending on the datasets available, different **sensitive features**, i.e., attributes of the dataset for which a machine learner may produce some form of discrimination, may lead to the emergence of bias toward specific **sensitive groups**.

It is not universally definable

Multiple fairness definitions available. Most are based on statistics, e.g., **predictive parity between sensitive and non-sensitive groups**, others on causal reasonings, e.g., **features graphs mapping inputs to outputs**.

It is, by nature, context-dependent

Context is king! A feature may be considered sensitive in one application domain but not in another, e.g., just think at gender discrimination in loan management versus medical diagnosis.

The Role of Software Engineering for Artificial Intelligence

Pre-processing algorithms

In-processing algorithms

Post-processing algorithms

The Role of Software Engineering for Artificial Intelligence

Pre-processing algorithms



Bias mitigation algorithms applied to training data

Fair Data Balancing



Fair Data Transformation

Dimensionality Reduction

Fairness metrics-driven data preprocessing

Fair-SMOTE algorithm removes biased labels; and rebalances internal distributions so that, based on sensitive attribute, examples are equal in positive and negative classes. models generated via Fair-SMOTE achieve higher performance (measured in terms of recall and F1) than other state-of-the-art fairness improvement algorithms.

Bias in Machine Learning Software: Why? How? What to Do?

Joymallya Chakraborty
jchakra@ncsu.edu
North Carolina State University
Raleigh, USA

Suvodeep Majumder
smajumd3@ncsu.edu
North Carolina State University
Raleigh, USA

Tim Menzies
timm@ieee.org
North Carolina State University
Raleigh, USA

ABSTRACT

Increasingly, software is making autonomous decisions in case of criminal sentencing, approving credit cards, hiring employees, and so on. Some of these decisions show bias and adversely affect certain social groups (e.g. those defined by sex, race, age, marital status). Many prior works on bias mitigation take the following form: change the data or learners in multiple ways, then see if any of that improves fairness. Perhaps a better approach is to postulate root causes of bias and then applying some resolution strategy.

This paper checks if the root causes of bias are the prior decisions about (a) what data was selected and (b) the labels assigned to those examples. Our Fair-SMOTE algorithm removes biased labels; and rebalances internal distributions so that, based on sensitive attribute, examples are equal in positive and negative classes. On testing, this method was just as effective at reducing bias as prior approaches. Further, models generated via Fair-SMOTE achieve higher performance (measured in terms of recall and F1) than other state-of-the-art fairness improvement algorithms.

To the best of our knowledge, measured in terms of number of analyzed learners and datasets, this study is one of the largest studies on bias mitigation yet presented in the literature.

CCS CONCEPTS

• Software and its engineering → Software creation and management; • Computing methodologies → Machine learning.

KEYWORDS

Software Fairness, Fairness Metrics, Bias Mitigation

ACM Reference Format:

Joymallya Chakraborty, Suvodeep Majumder, and Tim Menzies. 2021. Bias in Machine Learning Software: Why? How? What to Do? In Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAT* 2021), June 14–15, 2021, New York, NY, USA. ACM, 1–10. <https://doi.org/10.1145/3458714.3458715>

Sadly, there are too many examples of machine learning software exhibiting unfair/biased behavior based on some protected attributes like sex, race, age, marital status. For example:

- Amazon had to scrap an automated recruiting tool as it was found to be biased against women [1].
- A widely used face recognition software was found to be biased against dark-skinned women [2].
- Google Translate, the most popular translation engine in the world, shows gender bias. "She is an engineer, He is a nurse" is translated into Turkish and then again into English becomes "He is an engineer, She is a nurse" [3].

Prior works on managing fairness in machine learning software have tried mitigating bias by exploring a very wide space of control parameters for machine learners. For example, Johnson et al. [4] executed some (very long) grid search that looped over the set of control parameters of a learner to find settings that reduced bias. Another approach, published in FSE'20 by Chakraborty et al. [5] used stochastic sampling to explore the space of control options (using some incremental feedback operator to adjust where to search next). While these approaches were certainly useful in the domains tested by Johnson and Chakraborty et al., these methods are "dumb" in a way because they do not take advantage of domain knowledge. This is a problem since, as shown by this paper, such domain knowledge can lead to a more direct (and faster and more effective) bias mitigation strategy.

The insight we offer here is that the root causes of bias might be the *prior decisions that generated the training data*. Those prior decisions affect (a) what data was collected and (b) the labels assigned to those examples. Hence, to fix bias, it might suffice to apply mutators to the training data in order to

(A) remove biased labels; and (B) rebalance internal

The Role of Software Engineering for Artificial Intelligence

In-processing algorithms



Bias mitigation algorithms applied to the model itself during the training phase

Fairness Trade-Offs Mitigation



Generative Adversarial Training
Fair Graph Embedding Training

A novel ensemble approach to improving fairness-performance trade-off for ML software. Conventional ensemble methods combine different models with identical learning objectives. MAAT, instead, combines models optimized for different objectives: **fairness and ML performance**. MAAT significantly outperforms the state-of-the-art.

MAAT: A Novel Ensemble Approach to Addressing Fairness and Performance Bugs for Machine Learning Software

Zhenpeng Chen
zp.chen@ucl.ac.uk
University College London
London, United Kingdom

Federica Sarro
f.sarro@ucl.ac.uk
University College London
London, United Kingdom

Jie M. Zhang
jie.zhang@kcl.ac.uk
King's College London
London, United Kingdom

Mark Harman
mark.harman@ucl.ac.uk
University College London
London, United Kingdom

ABSTRACT

Machine Learning (ML) software can lead to unfair and unethical decisions, making software fairness bugs an increasingly significant concern for software engineers. However, addressing fairness bugs often comes at the cost of introducing more ML performance (e.g., accuracy) bugs. In this paper, we propose MAAT, a novel ensemble approach to improving fairness-performance trade-off for ML software. Conventional ensemble methods combine different models with identical learning objectives. MAAT, instead, combines models optimized for different objectives: fairness and ML performance. We conduct an extensive evaluation of MAAT with 5 state-of-the-art methods, 9 software decision tasks, and 15 fairness-performance measurements. The results show that MAAT significantly outperforms the state-of-the-art. In particular, MAAT beats the trade-off baseline constructed by a recent benchmarking tool in 92.2% of the overall cases evaluated, 12.2 percentage points more than the best technique currently available. Moreover, the superiority of MAAT over the state-of-the-art holds on all the tasks and measurements that we study. We have made publicly available the code and data of this work to allow for future replication and extension.

CCS CONCEPTS

the Foundations of Software Engineering (ESEC/FSE '22), November 14–18, 2022, Singapore, Singapore. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3540250.3549093>

1 INTRODUCTION

Software systems have been widely adopted to make decisions in social-critical, human-related tasks, including credit assessment [1], disease detection [55], criminal justice [4], and hiring [31]. The wide adoption of such decision-making software raises concerns about software fairness bugs (i.e., unfair software decisions) [27]. These bugs have been frequently reported, related to protected attributes such as race [5, 9] and sex [7, 17]. They may particularly disadvantage minorities and protected groups, resulting in unethical and unacceptable consequences.

The issue in fairness has been studied in both Software Engineering (SE) [40] and Machine Learning (ML) [62] research communities since 2008. From the SE perspective, fairness is a non-functional software property, as such can be the subject of testing to find fairness bugs [79]. In recent years, the surging ML software (i.e., software that relies on ML to tackle decision problems), increases the prevalence of software fairness bugs [58, 67]. Such prevalence attracts increasing attention from the SE community, and researchers have called for actions by the SE research to tackle fairness bugs

The Role of Software Engineering for Artificial Intelligence

Post-processing algorithms



Bias mitigation algorithms applied *a-posteriori* to the predicted labels

Generative Input Testing



Grammar-based Testing

Metamorphic Testing

BiasFinder curates templates based on the pieces of text from a large corpus. Next, it instantiates new text by filling in placeholders with words associated with a class of a characteristic (e.g., gender-specific words such as female names, “she”, “her”). These texts are used to tease out bias.

BiasFinder: Metamorphic Test Generation to Uncover Bias for Sentiment Analysis Systems

Muhammad Hilmi Asyrofi, Zhou Yang, Imam Nur Bani Yusuf, Hong Jin Kang, Ferdian Thung and David Lo

Abstract—Artificial Intelligence (AI) software systems, such as Sentiment Analysis (SA) systems, typically learn from large amounts of data that may reflect human biases. Consequently, the machine learning model in such software systems may exhibit unintended demographic bias based on specific characteristics (e.g., gender, occupation, country-of-origin, etc.). Such biases manifest in an SA system when it predicts a different sentiment for similar texts that differ only in the characteristic of individuals described. Existing studies on revealing bias in SA systems rely on the production of sentences from a small set of short, predefined templates. To address this limitation, we present BiasFinder, an approach to discover biased predictions in SA systems via metamorphic testing. A key feature of BiasFinder is the automatic curation of suitable templates based on the pieces of text from a large corpus, using various Natural Language Processing (NLP) techniques to identify words that describe demographic characteristics. Next, BiasFinder instantiates new text from these templates by filling in placeholders with words associated with a class of a characteristic (e.g., gender-specific words such as female names, “she”, “her”). These texts are used to tease out bias in an SA system. BiasFinder identifies a bias-uncovering test case (BTC) when it detects that the SA system exhibits demographic bias for a pair of texts, i.e., it predicts a different sentiment for texts that differ only in words associated with a different class (e.g., male vs. female) of a target characteristic (e.g., gender). Our empirical evaluation showed that BiasFinder can effectively create a larger number of fluent and diverse test cases that uncover various biases in an SA system.

Index Terms—sentiment analysis, test case generation, metamorphic testing, bias, fairness bug

1 INTRODUCTION

MANY modern software systems employ AI systems to make decisions. In AI systems, fairness is considered to be an important non-functional requirement; bias in AI systems, reflecting discriminatory behavior towards unprivileged groups, can lead to real-world harms. To address this requirement, software engineering research techniques, such as test generation, have been applied to detect bias [1]–[5]. While various techniques have been proposed for test generation of machine learning systems [1]–[4], there have been limited studies on detecting biases in text-based machine learning systems [5]. Text-based ML systems have numerous applications, for example, NLP techniques have been used for Sentiment Analysis (SA). It is, therefore,

as Microsoft^[1] and Google^[2] have developed and provided APIs for software developers to access SA capabilities. This suggests the prevalence of SA in real-life applications. As a result, bias in SA systems can have a big impact on society. Secondly, SA has generalizability to other areas of NLP. Some NLP researchers have considered SA to be “mini-NLP” [9], as research on SA techniques builds on top of a wide range of topics and tasks in the NLP domain. Cambria et al. [29] argues that SA is a problem with a composite nature, requiring 15 more fundamental NLP problems to be addressed at the same time. Therefore, we believe that tackling bias in SA is a suitable first step that could lead to a more general approach to detect bias in textual data.

Let's move to the code...

On the e-learning platform, you can find the code
we will discuss today!



sesa^{lab}
SOFTWARE ENGINEERING
SALERNO



Gianmario Voria

Ph.D. Student
Software Engineering (SeSa) Lab
University of Salerno